Banking, payments and one sign-in

Open accounts, issue cards, move money between banks and pay online. Independent banks connect through a secure hub, and customers use one account everywhere.

1 account	N banks	RSA + AES	3 stacks	No CDN
BANK AND SHOP	ONE HUB	EVERY BANK CALL	INDEPENDENT DEPLOYS	FONTS AND STYLES SERVED LOCALLY

WHAT A CUSTOMER DOES

Pay in the shop from a bank account

1. Choose a product and sign in once at the central login
2. Pick a bank at the hub: it opens the bank's confirmation page
3. Confirm with no second login: the money moves bank to bank
4. Back in the shop, the order is marked paid

- **Accounts and balance**
  Every account, balance and recent transfer on one dashboard.
- **Transfers**
  Inside the bank or to any bank on the hub, retried until the outcome is known.
- **Payment cards**
  Issue, edit and delete cards, then pay online with them.
- **Direct payments**
  Pay merchants from an account, with no card details.
- **Shop cabinet**
  Dashboard, orders, payment reminders and products with uploaded images.
- **Administration**
  All accounts, a transfer lookup and pending transfers for bank staff.

THREE INDEPENDENT STACKS

Bank
ASP.NET Core · SQL Server

- Bank web app
- CentralApi hub
- Kong gateway
- RabbitMQ retries
- nginx entry, rate limit

Shop
FastAPI · PostgreSQL

- React shop + cabinet
- Python API
- Redis sessions
- MinIO images

KYC
Keycloak · H2

- Login and register
- Edit profile
- Change password
- Single sign-on

Security built in: passwords live only in Keycloak, internal services publish no ports, public API calls are rate limited, and changing a profile or password signs the user in again.

UP AND RUNNING IN THREE STEPS

01
Start kyc
The central sign-in comes up at port 8080.

02
Start bank
The bank and the hub at port 8081.

03
Start shop
The shop and cabinet at port 8082.

Try it: sign in with demo@abs.uz · .NET 10, Python, React and TypeScript · Docker Compose · accelera-it/bank-system